

Projektdokumentation: Verbesserung des KI-Algorithmus für adaptive Gegner in einem Spiel.

13. Februar 2026

1 Ideenfindung

Die Idee zu finden, war nicht allzu schwierig. Schliesslich hatte meine Maturaarbeit etwas mit KI zu tun und bot interessante Anwendungsmöglichkeiten. Im Grunde ist die Idee dieselbe wie bei meiner Maturaarbeit: Gegner in einem Spiel zu programmieren, die aus den Handlungen der Spieler lernen können. Während meiner Maturaarbeit hatte ich ein Spiel programmiert und verschiedene KI-Algorithmen ausprobiert. In diesem Projekt wird ein weiterer Algorithmus, nämlich "Deep Q Learning", ausprobiert und anschliessend mit den bisherigen Algorithmen verglichen.

2 Hintergrund

Bevor mit der eigentlichen Implementierung begonnen wird, wird zunächst etwas Kontext zum erstellten Spiel und den genutzten Algorithmen gegeben.

2.1 Spiel und Gegner

Das Spiel lässt sich als 2D-Top-Down-Rogue-like-Shooter klassifizieren. Was sich genau dahinter verbirgt, ist in Abbildung 1 dargestellt. Für dieses Projekt sind die Gegner im Spiel von Bedeutung.

Die Gegner sind kleine grüne Kreaturen, die den Spieler angreifen. Dafür stehen ihnen fünf mögliche Aktionen zur Verfügung, die im Code 1 zu finden sind.

Code 1: Mögliche Aktionen

```
1  var valid_actions = ["move_forward", "strafe_left", "strafe_right", "retreat", "use_weapon"]
```

Sie erscheinen in Wellen, das heisst, eine bestimmte Anzahl von ihnen erscheint nacheinander. Wenn sie sterben, kommen neue Gegner nach. Dabei sollten die Gegner sowohl während



Abbildung 1: Gameplay des Spiels

der Wellen als auch zwischen ihnen aus den Spielern lernen. Dafür wurde ein spezifischer Algorithmus entwickelt, der im nächsten Abschnitt erläutert wird.

2.2 Frühere Implementierung des adaptiven KI

Der beim Matura-Spiel genutzte Algorithmus wurde Genetisches Q-Learning genannt. Dabei handelt es sich um einen Hybrid-Algorithmus, der genetische Algorithmen und Q-Learning verbindet. Während der Welle ist Q-Learning für das Lernen zuständig. Dadurch werden Taktiken wie die Positionierung, das Ausweichen der Spielerkugel, das richtige Angriffszeitfenster und die Tatsache, dass die Distanz zum Spieler reduziert werden muss, damit der Gegner angreifen kann, gefunden.

Für das Lernen zwischen den Wellen war der genetische Algorithmus verantwortlich. Nach jeder Welle wurden die Gegner anhand ihrer Fitness bewertet und die besten wurden mit Mutation reproduziert. So erscheinen die Gegner mit besseren Q-Tabellen in jeder Welle.

Das neue Algorithmus wird dabei Q-Learning verbessern. Das Lernen zwischen der Wellen, respektiv Genetische Algorithmen wird daher in diesem Projekt nicht relevant und nicht bewertet.

Q-Learning und Deep-Q-Learning sind Algorithmen aus dem Bereich des Reinforcement Learnings. Da wir uns nur mit Supervised Learning befassen haben, verfügen wir über keine Kenntnisse dieses Algorithmus. In den nächsten Abschnitten werden deshalb die Algorithmen erklärt.

2.3 Q-Learning

Mit Q-Learning handelt es sich um eine modellfreie Methode, die eine optimale Strategie lernt, indem sie die Q-Wert-Funktion schätzt. Diese gibt an, wie vorteilhaft eine bestimmte Aktion in einer bestimmten Situation ist. Das Ziel besteht darin, einen Plan zu finden, der im Laufe der Zeit die höchste Gesamtbelohnung bringt. Das Schema in Abbildung 2 visualisiert die Funktionsweise des Algorithmus. [1]

Um dies besser zu verstehen, betrachten wir ein Beispiel aus dem Spiel. Zunächst wird der Zustand berechnet. Dieser kann beispielsweise wie folgt aussehen: „wt0d4a0bd-1ba-1“. In ihm

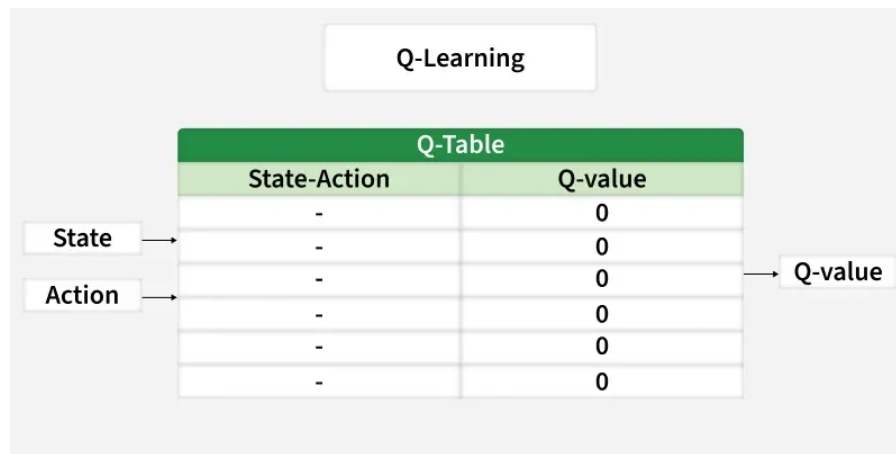


Abbildung 2: Tabulares Q-Learning

sind die Daten über die Umwelt des Gegners kodiert. „d4“ bedeutet „Distanz“-4, das heisst, die Distanz zum Spieler ist gross. Mithilfe dieses Zustands wird in der Tabelle die beste Aktion (die Aktion mit dem höchsten Q-Wert im Spiel) gefunden. Ein Eintrag für einen Zustand in der Q-Tabelle sähe zum Beispiel wie folgt aus:

```
{ ...
"wt0d4a0bd-1ba-1": { "move_forward": 4.3, "strafe_left": -2.5,
"strafe_right": -1.4, "retreat": -5.7, "use_weapon": 2 },
... }
```

2.4 Deep-Q-Learning

Q-Learning funktioniert gut bei einem begrenzten Zustandsraum. Wenn jedoch der Zusammenhang zwischen mehreren Features ermittelt werden soll und die Werte für die Features dabei kontinuierlich angegeben werden müssen, gibt es ein Problem. Der Zustandsraum wird zu gross und die KI gerät nie in denselben Zustand, was Lernen unmöglich macht. Um dies zu lösen, könnte man statt der Tabelle eine Funktion verwenden, die den Q-Wert schätzt. Dabei müsste statt der Tabelle die Funktion gemäss den Belohnungen geändert werden. Genau das tut der Deep-Q-Learning-Algorithmus mithilfe eines neuronalen Netzwerks.

Der Zustand kann dabei beispielsweise als Dictionary mit Feature-Wert-Paaren oder als geordneter Array mit Werten dargestellt werden. Die Werte werden als Input an ein neuronales Netz übergeben. Als Output erhält man die Q-Werte für jede Aktion. Wie beim tabellarischen Q-Learning wird die Aktion mit dem höchsten Q-Wert ausgewählt und in der Umgebung ausgeführt. Anschliessend wird die Belohnung vergeben, was im Fall von Deep Q-Learning die Backpropagation ausführt. Das Schema in Abbildung 3 visualisiert die Funktionsweise des Algorithmus. [1]

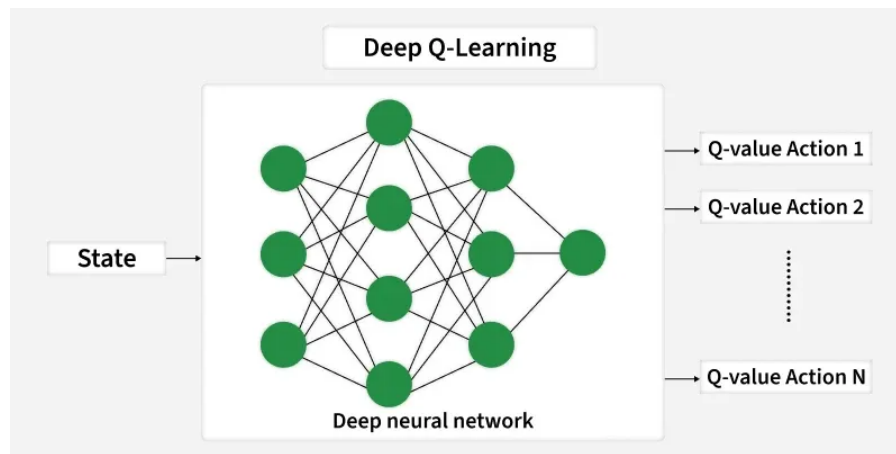


Abbildung 3: Deep Q-Learning

3 Datensatz

Da sich das Projekt mit Reinforcement Learning befasst, spricht man nicht wirklich von einem Datensatz. Das Modell erhält lediglich Informationen über seine Umwelt, ohne die besten möglichen Aktionen zu kennen: Es gibt keine Labels wie beim Supervised Learning. Die Daten, die die Umwelt des Modells beschreiben, heissen Zustand. Welche Parameter (Features) es genau enthält, muss bewusst gewählt werden.

3.1 Auswahl der Features

Die Parameter des Zustands sind in Code 2 zu sehen. Alle Parameter ausser den ersten beiden haben kontinuierliche Werte, die beiden ersten haben dagegen diskrete Werte. Sie wurden so gewählt, dass die Gegner ihre Taktik je nach verwendeter Waffe (Nahkampf oder Fernkampf) ändern können. Andere Parameter sollen vor allem das richtige Positionieren und Angreifen ermöglichen.

Code 2: Features im Zustand

```

1  self.STATE_KEYS = [
2      "weapon_type",
3      "player_weapon_type",
4      "pos_x",
5      "pos_y",
6      "dist_to_player",
7      "angle_to_player",
8      "bullet_dist",
9      "bullet_angle",
10     "dist_ally",
11     "angle_ally"
12 ]

```

3.2 Aufbereitung & Aufteilung des Datensatzes

In jedem Frame des Spiels werden die Daten des Gegners von Godot geholt und als JSON über ein Socket an Python geschickt. Dabei stellt der Zustand ein Dictionary dar, dessen Keys die Parameter aus Code 2 beinhalten und dessen Werte die von Godot gesammelten Daten sind. Es wurde auch überlegt, den Zustand als Array zu schicken, man entschied sich jedoch aus Gründen der Klarheit der Daten dagegen.

Im Gegensatz zum Supervised Learning ist hier darüber hinaus keine Trennung in Trainings- und Testdaten erforderlich. Um die Performance zu testen, wird das Modell stattdessen eingefroren, die Epsilon-Exploration ausgeschaltet und das Ergebnis einiger Episoden beobachtet.

4 Implementierung des KI-Modells

4.1 Überblick

Das Ziel des Modells besteht darin, auf Basis des berechneten Zustands die beste Aktion jedes Gegners zu ermitteln. Darüber hinaus, damit das globale Ziel der Adaptivität der Gegner sichtbar wird, soll das Modell nicht sofort optimierte Parameter haben, sondern diese sollen im Laufe des Spiels gefunden werden. Dieser Ansatz kann dem Online Machine Learning zugeordnet werden, was bedeutet, dass das Training nie wirklich endet.

Das Modell ist im Code als Klasse „DQNLearner“ dargestellt. Es besteht aus zwei neuronalen Netzen mit gleicher Architektur, die im Code 3 beschrieben sind. Das erste neuronale Netz heisst „Online DQN“ und bestimmt die Aktionen der Agenten. Das zweite Netz heisst Target DQN und evaluiert diese Aktionen. Target DQN ist eine einfache, eingefrorene Kopie von Online DQN, die periodisch aktualisiert wird.

Code 3: Die Architektur des neuronalen Netzes

```
1  class QNetwork(nn.Module):
2      def __init__(self):
3          super().__init__()
4          self.net = nn.Sequential(
5              nn.Linear(10, 32), # layer 1 (hidden)
6              nn.ReLU(),
7              nn.Linear(32, 32), # layer 2 (hidden)
8              nn.ReLU(),
9              nn.Linear(32, 5) # (layer 3 -> output)
10         )
11
12     def forward(self, x):
13         return self.net(x)
14
```

4.2 Modeltraining

Der Trainingsprozess und die Funktionsweise des Modells werden wie folgt aussehen:

1. Das Spiel sendet den Zustand als Dictionary mit Feature-Werten für die jeweiligen Gegner.
2. Das Modell berechnet die Q-Werte jeder Aktion mithilfe eines neuronalen Netzes und sendet die Aktionen mit den höchsten Werten zurück an das Spiel. Die Architektur des Netzwerks und dessen Hyperparameter sind in Abbildung 1 dargestellt.
3. In der Umgebung des Spiels werden diese Aktionen ausgeführt. Das Spiel schickt basierend auf dem Geschehen im Spiel voreingestellte Belohnungen zurück.
4. Mithilfe dieser Belohnungen kann ein Trainingsschritt durchgeführt werden. Die Parameter, die die durchgeführte Aktion beeinflusst haben, können mittels Backpropagation geändert werden. Um zu bestimmen, wie die Parameter genau geändert werden sollen, muss der Verlust bekannt sein. Dieser wird durch MSE vom Q-Wert des Online-Netzwerks und einem „Zielwert“ berechnet. Dieser Wert wiederum wird durch die Belohnung und den Q-Wert für die Aktion des Target-Netztes bestimmt.
5. Mit dem geführten Trainingsschritt beginnt eine neue Episode und die Gegner wechseln in einen neuen Zustand.

Dieser Prozess wiederholt sich bei jedem Frame des Spiels. Er läuft mit 60 fps, d. h., die Episode dauert eine 60stel Sekunde. Es wurde auch überlegt, den Trainingsschritt nicht bei jedem Frame auszuführen, um das Lernen zu stabilisieren und die Spielleistung zu optimieren. Dafür könnte man einen Replay-Buffer schaffen, in dem einzelne Batches mit Zuständen und Aktionen gespeichert werden, aus denen dann periodisch gelernt wird. Der Grund, warum Replay-Buffer nicht implementiert wurde, ist vor allem Zeitmangel.

4.3 Modellbewertung

Um die Modelle zu bewerten, wird im Spiel eine Umgebung geschaffen, die eher einer Simulation als einem Spiel gleicht. In dieser Umgebung wird der Spieler durch einen Bot mit einfachen Verhaltensmustern ersetzt, sodass es für die Gegner einfacher ist, zu lernen. Darüber hinaus wird eine an das Spiel angepasste Metrik namens „Fitness“ eingeführt. „Fitness“ ist proportional zur Lebenszeit, dem verursachten Schaden und den ausgewichenen Kugeln. Während des Spiels wird die Fitness im Zeitverlauf gemessen, wodurch die Effizienz des Modells bewertet wird.

4.4 Anwendung des Modells

Die Anwendung des Modells ist direkt auf das Ziel ausgerichtet. Es soll das Verhalten der Gegner im Spiel bestimmen. Da es sich um RL handelt, könnte das Modell darüber hinaus nicht ohne Umgebung funktionieren. Das Spiel selbst wurde früher durch Godot geschaffen, in diesem Projekt wurde diese kompiliert, sodass keine Godot-Installation nötig ist.

5 Ergebnisse und Vergleich

5.1 Qualitative Bewertung

Mit dem Deep-Q-Learning-Verfahren scheinen sich die Gegner weniger aggressiv zu verhalten und halten dabei auch grosse Distanz zum Spieler. Manchmal weichen sie Kugeln aus, aber nicht immer. Es wirkt, als würden sie das Ausweichen für einen Moment erlernen, es kurz darauf jedoch wieder vergessen. Dieses Verhalten ist ein klassisches Problem des Online-ML und heisst „catastrophic interference“ (den Namen habe nicht ich erfunden). Insgesamt ist das Verbesserungspotenzial deutlich erkennbar.

5.2 Quantitative Bewertung

Die Ergebnisse des Experiments sind in Abbildung 4 dargestellt.

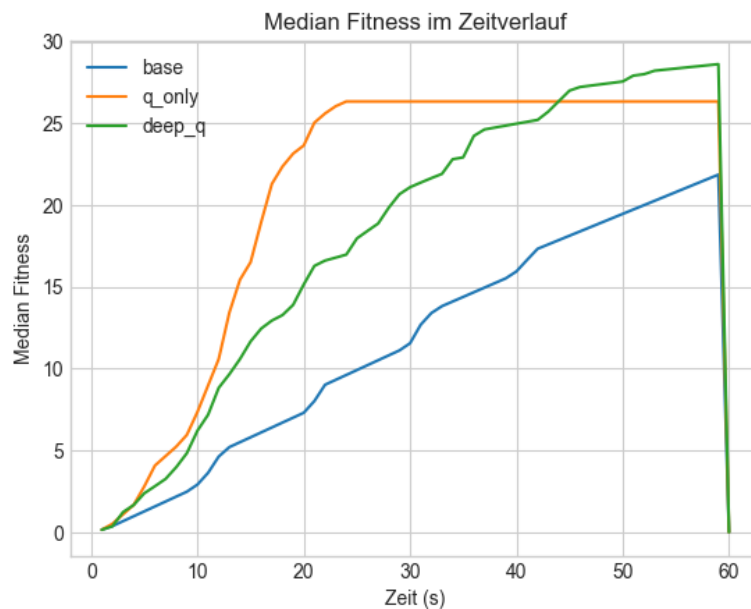


Abbildung 4: Fitness vs. Zeit-Plot

In der Abbildung sind drei Konfigurationen dargestellt:

1. „base“: Konfiguration mit zufälligem Gegnerverhalten
2. „q_only“: Konfiguration mit altem, tabellarischem Q-Learning.
3. „deep_q“: Konfiguration mit dem neuen, im Projekt probierten Deep Q-Learning-Algorithmus.

Wie man sieht, führt zufälliges Verhalten logischerweise zu den schlechtesten Ergebnissen. Der alte Algorithmus scheint zwar anfangs eine höhere Fitnesssteigerung zu bewirken, jedoch zeigt sich das Problem dieser Konfiguration bereits nach ca. 20 Sekunden. Die Gegner sind zu aggressiv, sterben und führen so zu einer konstanten, wenn auch hohen Fitness. Gegner, die nach dem Deep-Q-Algorithmus handeln, haben zwar eine niedrigere Fitnesssteigerung, leben jedoch länger, was zu einer höheren Gesamtfitness führt. Insgesamt scheinen die Gegner deswegen die Zukunft mehr zu berücksichtigen und in gewisser Weise schlauer zu sein.

5.3 Ausblick

Um das Modell zu verbessern, könnte man noch einiges tun:

1. Replay-Buffer. Dadurch könnte das Problem, dass die Gegner ihre neu erlernten Taktiken vergessen, höchstwahrscheinlich behoben werden.
2. Epsilon Decay. Der Epsilon-Wert gibt an, wie oft eine zufällige Aktion für die Exploitation gewählt wird. In diesem Projekt ist er konstant gehalten. Viele RL-Modelle setzen ihn am Anfang sehr hoch und senken ihn langsam, damit die Agenten nicht zu schnell das gleiche Verhalten zeigen. Epsilon Decay könnte das Modell wahrscheinlich auch in gewissem Masse verbessern, jedoch nicht im selben Ausmass wie der Replay Buffer.

5.4 Fazit

Insgesamt lässt sich sagen, dass das Projekt relativ erfolgreich war. Der Deep-Q-Learning-Algorithmus wurde mit dem Spiel verbunden und die Unterschiede zwischen dem neuen und dem alten Modell wurden analysiert. Meine persönliche Reflexion: Das KI-Projekt hat mir Spass gemacht und ich habe viel gelernt. Allerdings habe ich ziemlich viel KI genutzt, um KI zu schaffen (siehe KI-Nutzung).

Anhang

Tabelle 1: Experimentalmatrix: Übersicht der Messgrössen

Versuchsvariablen	Werte / Beschreibung
Repeats	$N_{rep} = 3$
Seed-Strategie	Je repeat ein neuer Zufallssamen; protokolliert für Reproduzierbarkeit
Logging	Pro Gegner: damage, lifespan, fitness; pro Welle: sekundliche Snapshots (mean_fitness_alive)
Bewertungsfunktion	$f(L, D, B) = 7D + 0.2L + 1B$
Anzahl Gegner pro Welle	6
Wellenzeitlimit	60 sec

Tabelle 2: Verwendete Hyperparameter in den Experimenten.

Kategorie	Parameter	Wert / Beschreibung
Q-Learning	Lernrate (α)	0,2
	Diskontierungsfaktor (γ)	0,9
	Explorationsrate (ϵ)	0,1
	Anzahl der Hidden Neurons	32 jeweils in 2 Layers
Belohnungen	Treffer auf Spieler	+10,0
	Eigener Schaden	-5,0
	Überlebenszeit	+0,0 pro Sekunde
	Tod	-5,0
	Bewegung ohne Nutzen	-0,05
	Rückzug	-0,2
	Fehlgeschlagener Schuss	-0,2
	Bewegung in Richtung Spieler	+0,05
	Feststecken	-1,0
	Ausweichmanöver	+6,0

Literatur

[1] GeeksforGeeks. Deep q-learning in reinforcement learning, 2025. Last updated 09 Oct 2025, accessed 07 Jan 2026.

- Künstliche Intelligenz. Alle Chats sind unter dem folgenden Link zu finden: <https://chatgpt.com/g/g-p-6939856e87908191a3fc2288cb297f64-deep-q-learning-game/project> Die konkrete Nutzung ist in Tabelle 3 dokumentiert.
- Git und GitHub (<https://github.com/Jokolto/deep-q-learning-enemies>)
- Eigene Vorarbeiten aus der Maturaarbeit (<https://github.com/Jokolto/Matura>)

KI-Tool	Einsatzform	Betroffene Teile des Codes	Bemerkungen
ChatGPT	Vertiefung des Verständnisses zum Thema und Schaffung eines Prototyps.	deepq_learner.py	Der generierte Code wurde speziell für das Spiel angepasst.
DeepL Write	Überarbeitung der geschriebenen Dokumentation	Latex-Dokumentation	keine

Tabelle 3: Einsatz verschiedener KI-Tools im Projekt